

CS-3510-B Problem Set 2

Elijah Martin McCauley

TOTAL POINTS

74 / 80

QUESTION 1

1 Divide and Conquer 20 / 20

✓ - **0 pts** *Correct*

- **17 pts** Missing or incorrect algorithm
- **15 pts** Algorithm is mostly incorrect/ not using D&C
- **5 pts** Algorithm is not efficient/ partially correct
- **5 pts** Missing or incorrect time complexity
- **5 pts** Proof of Correctness missing
- **3 pts** Minor mistake in algorithm
- **20 pts** No answer
- **10 pts** Incomplete Algorithm

QUESTION 2

2 Divide and Conquer 15 / 20

- **0 pts** Correct
- **17 pts** Missing or incorrect algorithm
- **15 pts** Algorithm is mostly incorrect/ not using D&C
- **10 pts** Algorithm is not efficient/ partially correct
- **7 pts** Pseudo-code provided, but algorithm is missing.
- **5 pts** Missing or incorrect time complexity
- ✓ - **5 pts** *Proof of Correctness missing*
- **3 pts** Minor mistake in algorithm
- **20 pts** No answer

QUESTION 3

3 Modular Arithmetic 9 / 10

- **0 pts** (a) Correct(100) and work shown
- **1 pts** (a) Correct answer, but no work shown

OR

Incorrect answer, but made good attempt

- **2 pts** (a) Incorrect and no work shown
- **0 pts** (b) Correct(3) and work shown

✓ - **1 pts** (b) *Correct answer, but no work shown OR Incorrect answer, but made good attempt*

- **2 pts** (b) Incorrect and no work shown
- **0 pts** (c) Correct(9) and work shown
- **1 pts** (c) Correct answer, but no work shown

OR

Incorrect answer, but made good attempt

- **2 pts** (c) Incorrect and no work shown
- **0 pts** (d) Correct(20) and work shown
- **2 pts** (d) Correct answer, but no work shown

OR

Incorrect answer, but made good attempt

- **3 pts** (d) Incorrect and no work shown

QUESTION 4

RSA Cryptosystem 20 pts

4.1 Part A 15 / 15

- ✓ - **0 pts** (a) *Correct (Stephen)*
- **5 pts** (a) Incorrect (Malav)

✓ - 0 pts (b) Correct answer (64) with complete justification.

- 2 pts (b) No simplification provided for mod

- 3 pts (b) Correct answer (64) with insufficient justification.

- 3 pts (b) Incorrect answer with significant progress

- 5 pts (b) Incorrect answer with little/no progress.

✓ - 0 pts (c) Fully Correct

- 1 pts (c) Minor Error

- 3 pts (c) Incomplete, insufficient, or partially correct answer

- 5 pts (c) Completely incorrect answer

4.2 Part B 5 / 5

✓ - 0 pts Correct

- 2 pts Incorrect justification for why the encryption is insecure.

- 2 pts Encryption suggestion too vague and can't be implemented

- 2.5 pts Missing suggestion

- 5 pts Completely incorrect / Missing

QUESTION 5

5 Fast Fourier Transform (FFT) 10 / 10

✓ - 0 pts (a) Correct

$f: (14, 9 + 3i, 8, 9 - 3i)$

$g: (-2, 1 - 3i, 4, 1 + 3i)$

$h: (-28, 18 - 24i, 32, 18 + 24i)$

- 2 pts (a) FFTs correct; No work shown

- 2 pts (a) Work shown, but no final answer

- 2 pts (a) FFT wrong for $f(x)$

- 2 pts (a) FFT wrong for $g(x)$

✓ - 0 pts (b) Correct

$h(x) = 10 - 27x - 8x^2 - 3x^3$

$h: (-28, 18 - 24i, 32, 18 + 24i)$

- 3 pts (b) Answer is correct, but no work shown

- 2 pts (b) Answer is wrong, but consistent with

Part A

- 2 pts (b) Work is correct, but final answer

missing

- 7 pts Both (a) and (b) are wrong , but good attempt

- 10 pts Incorrect/Missing Answer

Name: Elijah McCauley

1.) Divide and Conquer (20 points)

An alien landed on our planet from a parallel universe and brought a new definition of string equivalency. They say that two equal-length strings p and q are equivalent in one of the following way:

- (a.) Strings p and q are equal.
- (b.) If string p is split into two parts p_1 and p_2 , each of the same length, and string q into two parts q_1 and q_2 , each of the same length, then one of the following is correct:
 - (a) p_1 is equivalent to q_1 , and p_2 is equivalent to q_2 .
 - (b) q_1 is equivalent to p_2 , and p_1 is equivalent to q_2 .

As a CS 3510 student, assuming the lengths of p and q are powers of 2, help the alien design a Divide & Conquer algorithm that runs specifically in $\mathcal{O}(N \log N)$ time complexity (where N represents the length of the two strings) to determine if two given strings are equivalent in his/her world. Make sure to provide proof of the correctness of your algorithm and the time complexity (using Master's Theorem).

Examples:

Input: $p = aabb$, $q = bbaa$

Output: True

Explanation: The second condition in b. holds true.

Input: $p = abbb$, $q = bbba$

Output: True

Explanation: The second half of p is equal to the first half of q . For the first half of p , $p_1 = "ab" = 'a' + 'b'$ and the second half of q , $q_2 = "ba" = 'b' + 'a' = 'a' + 'b'$. Hence, p_1 is equivalent to q_2 .

Input: $p = aabb$, $q = baab$

Output: False

Explanation: None of the conditions hold.

Solution: The first thing you need to do is see if the first equivalence check (a. Strings p and q are equal) works by directly comparing the two strings. This is done by simply comparing the strings p and q letter by letter in $\mathcal{O}(n)$ time and recording the letters as you go.

If the two strings have the same letters but are not in the exact same order, you need to proceed with a version of merge sort. Starting with p , we must split it up into 2 equal parts and then take those substrings which we will call p_1 and p_2 , and split them into two equal parts. We need to repeat this process until we have reached the base case where each substring is now length 1. Now that we have a list that we can call L containing N

strings, we must begin the process of combining them. For each pair of strings in the array (string 1 and 2, string 3 and 4, etc.), we need to compare the first letter of the string and if they are different we order the two strings lexicographically based on the first letter and combine them into one string of double length. If the two characters are equal we move on to comparing the next letters lexicographically until we reach the end of the two and simply combine while maintaining order because they are equal or find different letters and order the strings lexicographically based on the first differing letter. Once those two strings are combined we move on to the next two strings and order and combine them using that same process. We repeat this process with all the strings in L until the length of L has been halved (meaning all the strings have been combined with another). We then repeat this process of combining pairs of strings until we have reached only 2 strings remaining of size $N/2$ which we can call p_1 and p_2 . The time complexity for this process can be found using Master's Theorem. The branching factor is $a = 2$ since we recurse on two substrings, the size of subproblem is $b = 2$ because each recursion is on a substring of half the previous size, and the cost to recombine is $\mathcal{O}(n)$, so $d = 1$. $T(n) = 2T(n/2) + \mathcal{O}(n)$ Using master's theorem, $d = 1 = \log_b(a) = 1$, so the time complexity is $\mathcal{O}(n \log(n))$.

We then repeat this whole process on string q . Splitting it up into equal length parts until we have made a list of N strings of length 1. Combining the strings by comparing two at a time and ordering them lexicographically before appending one to the other. Repeating this previous step until the list only contains two substrings of length $N/2$ which we will call q_1 and q_2 . This part has the same time complexity as the previous function because we again break it into $a = 2$ recursive parts of size $n/2$ so $b = 2$ and the cost to recombine is $\mathcal{O}(n)$ so $d = 1$. Again $T(n) = 2T(n/2) + \mathcal{O}(n)$, so by Master's Theorem $d = 1 = \log_b(a) = 1$, so its runtime is $\mathcal{O}(n \log(n))$.

Finally, we must check the alien's second definition of equivalency. We compare p_1 and q_1 letter by letter to see if they are equal. If they are, we compare p_2 and q_2 the same way. If both $p_1 = q_1$ and $p_2 = q_2$ then the strings are equivalent by definition (b.) (a). If either of the last two checks fail, we now compare q_1 and p_2 letter by letter and if they are equal we compare p_1 and q_2 letter by letter to see if they are equal. If both $q_1 = p_2$ and $p_1 = q_2$ then the strings are equivalent by definition (b.) (b). If both of these checks fail, then the strings are not equivalent. These final checks are $\mathcal{O}(n)$ time because they loop through n at most 4 times which is still $\mathcal{O}(n)$.

The runtime overall is $\mathcal{O}(n \log n)$ because the two variations of merge sort that we run on p and q are the most time costly part of the algorithm (compared to the initial and final string checks in $\mathcal{O}(n)$ time).

1 Divide and Conquer 20 / 20

✓ - 0 pts *Correct*

- 17 pts Missing or incorrect algorithm
- 15 pts Algorithm is mostly incorrect/ not using D&C
- 5 pts Algorithm is not efficient/ partially correct
- 5 pts Missing or incorrect time complexity
- 5 pts Proof of Correctness missing
- 3 pts Minor mistake in algorithm
- 20 pts No answer
- 10 pts Incomplete Algorithm

2.) Divide and Conquer (20 points)

Our wonderful TA, Bharat, has C cats and H cat houses where $C < H$. These cat houses are located in a straight line, and the positions are given as an array A of 1D coordinates (the coordinates are guaranteed to be in strictly increasing order i.e. A is given in a sorted order). Bharat is concerned about cat fights in his house, if the cats are placed too closely. To prevent the cats from hurting each other, Bharat wants to assign cats to their houses, such that the minimum distance between any two of them is as large as possible.

You are given a black-box algorithm called $placeCats(A, d, C)$. It takes in three parameters: the position array A , the distance d that you would like to test, and the number of cats C . In $O(H)$ time, it outputs a boolean indicating whether there's a placement of cats such that the minimum distance between any two cats from C cats is d .

Example:

Input: $H = 5$, $C = 3$, $A = [1, 2, 4, 8, 9]$

Output: 3

Explanation:

We want to choose 3 of the houses (for $C = 3$ cats) out of $[1, 2, 4, 8, 9]$ so that the minimum distance between any two of them is maximal. For this case, we can place the cats at position 1, 4 and 9, and the minimum distance between any of the two cats will be 3 as $\min(4 - 1, 9 - 4) = 3$.

Why not 4? If we place the first cat at position 1, the second cat at position 8 and the third cat at position 9, we will get $\min(8 - 1, 9 - 8) = 1$. Hence, there's no way we can get a minimum distance of 4.

Why not 2? If we place the first cat at position 2, the second cat at position 4 and the third cat at position 9, we will get $\min(4 - 2, 9 - 4) = 2$. But, this is not the largest of all the minimum distances. Hence, 3 is the largest minimum distance that the cats can be placed.

As a CS 3510 student, please help Bharat design an algorithm that outputs the largest minimum distance that the cats can be assigned a house on the given location array A . The time complexity of your solution should be $O(H \log M)$ where H is the number of cat houses and M is the max difference of array A i.e. $\max(A) - \min(A)$. Make sure to provide the proof of correctness of your algorithm. Also, please give a brief intuitive justification for the runtime (proof using Master's Theorem is not required).

Hint: Compute the maximum separation from array A , and then try to optimize your search for distances to achieve the given time complexity.

Solution: In order to compute the maximum distance between the cats, you need to use a variation of binary search. This search starts by getting the minimum ($A[1]$) and maximum ($A[A.length]$) values of the array as well as the distance between the two by taking maximum - minimum. You then set a left variable to be equal to the min value and a right variable

to be equal to the maximum value. While the left variable is $\leq$ the right variable, you find the mid value using $(left + right) \div 2$, you run `placeCats(A, mid, C)`, and if it returns true, update the left to equal `mid + 1`, and if its false set right equal to `mid - 1`. We then repeat that process using the new left and right variables to recalculate mid, run `placeCats`, and update left or right. When the while loop breaks out, meaning left is now greater than right, you want to return `left - 1` as the maximum distance between the cats given the array A, the number of cats C, and the number of houses H. `Left - 1` is returned because we are breaking out of the loop when Left is greater than right; if the previous mid value tested was true, then left will become `mid + 1` before breaking out of the loop, so `Left - 1` will be that previous mid that was just successful. If the loop breaks out on a false test, then right becomes `mid - 1` making it less than left (we know because it broke out), so left must equal mid which failed so we return `left - 1` which was the last successful test because left only updates to `mid + 1` on a success. Basically, left updates to `mid + 1` on a successful test, so when we breakout and return `left - 1`, we are returning the distance used on the last successful test.

My brief justification for the runtime of the function being $\mathcal{O}(H \log M)$ where H is the number of Houses and M is the maximum distance of array A, is that the `placeCats` method that we run is in $\mathcal{O}(H)$ time, and because we are checking always checking the middle value, starting at M we are halving the number of elements we are looking at each iteration which results in $\log(M)$ iterations. This means we run `placeCats` $\mathcal{O}(\log(M))$ times making the overall function $\mathcal{O}(H \log M)$.

2 Divide and Conquer 15 / 20

- **0 pts** Correct
- **17 pts** Missing or incorrect algorithm
- **15 pts** Algorithm is mostly incorrect/ not using D&C
- **10 pts** Algorithm is not efficient/ partially correct
- **7 pts** Pseudo-code provided, but algorithm is missing.
- **5 pts** Missing or incorrect time complexity
- ✓ - **5 pts** *Proof of Correctness missing*
- **3 pts** Minor mistake in algorithm
- **20 pts** No answer

3.) Modular Arithmetic (10 points)

Use Fermat's Little Theorem to evaluate the following (all moduli are prime):

(a.) $100^{199} \pmod{199}$

(b.) $4^{63} \pmod{61}$

(c.) $9^{82} \pmod{163}$

(d.) $3^{176} \pmod{179}$

Solution: (a.) Using Fermat's Little Theorem, $100^{198} \cong 1 \pmod{199}$ and $100 \cdot 100^{198} \cong 100 \cdot 1 \pmod{199}$, so $100^{199} \pmod{199} = 100$.

(b.) Using Fermat's Little Theorem, $4^{60} \cong 1 \pmod{61}$ and $4^3 \cdot 4^{60} \cong 4^3 \cdot 1 \pmod{61}$, so $4^{63} \pmod{61} = 64$.

(c.) Using Fermat's Little Theorem, $3^{162} \cong 1 \pmod{163}$ and $3^2 \cdot 3^{162} \cong 3^2 \cdot 1 \pmod{163}$, so $3^{164} \pmod{163} = 9^{82} \pmod{163} = 9$.

(d.) Using Fermat's Little Theorem, $3^{178} \cong 1 \pmod{179}$. We need to multiply the left side by 3^{-2} to get that side to be 3^{176} . In order to do that we need to find 3's modular inverse for 179 which $3 \cdot 60 \pmod{179} \cong 1 \pmod{179}$, so using modular inverses, $3^{-1} \cong 60 \pmod{179}$. Since we multiply the left side of the congruency by 3^{-2} , we multiply the right side by $60^2 \pmod{179}$. This results in $3^{176} \cong 60^2 \pmod{179} = 20$.

3 Modular Arithmetic 9 / 10

- 0 pts (a) Correct(100) and work shown

- 1 pts (a) Correct answer, but no work shown OR

Incorrect answer, but made good attempt

- 2 pts (a) Incorrect and no work shown

- 0 pts (b) Correct(3) and work shown

✓ - 1 pts (b) Correct answer, but no work shown OR

Incorrect answer, but made good attempt

- 2 pts (b) Incorrect and no work shown

- 0 pts (c) Correct(9) and work shown

- 1 pts (c) Correct answer, but no work shown OR

Incorrect answer, but made good attempt

- 2 pts (c) Incorrect and no work shown

- 0 pts (d) Correct(20) and work shown

- 2 pts (d) Correct answer, but no work shown OR

Incorrect answer, but made good attempt

- 3 pts (d) Incorrect and no work shown

4.) RSA Cryptosystem (20 points)

Part A

Suppose Malav wants to send a secured email to Stephen using the RSA scheme. Help your TAs answer these questions.

- (a.) Who should generate the RSA public and private keys? What are the quantities that the other party will need in order to generate an encrypted message?
- (b.) Suppose the person generating the key chooses the primes $p = 29$ and $q = 7$ and computes $e = 9$. Suppose the message that Malav sent is $m = 9$, what's the encrypted message received by Stephen? You may use calculator for this part, but show your work for full credit.
- (c.) Now suppose, Malav wants to send a secured email to both Stephen and Rohit. Normally, Malav would generate a unique public, private key pair for each, but he has bad randomness. Instead of generating four unique primes, two for each, he only generates three. He constructs the moduli for the two public keys as $N_1 = pq$ and $N_2 = qr$. The public exponents are chosen normally. Show this is insecure. Explain your response.

Solution: Stephen should generate both the public and private keys because the private key is needed to decypher but not needed to encrypt the message: In order to generate an encrypted message, Malav will only need the two parts of the public key: n (the modulus which equals the product of two large prime numbers) and e (the public exponent which is an integer relatively prime to the totient of n)

Using the two primes $p = 29$ and $q = 7$, we can multiply them together to get n : $n = p * q = 29 * 7 = 203$. We can then use the given $e = 9$ and $m = 9$, to plug into the equation: $c = m^e \pmod{n}$. This solves out to $c = 9^9 \pmod{203} = 64$

It is insecure because if a person has access to both the public keys N_1 and N_2 and they know that there is only 3 unique primes, then they can find the greatest common factor of N_1 and N_2 which will be q . From there they can easily divide N_1 by q to get p and N_2 by q to get r . Because e is a public key, we now have e_1, e_2, p, q , and r , which allows us to use $e_1^{-1} \pmod{(p-1)(q-1)}$ and $e_2^{-1} \pmod{(r-1)(q-1)}$ to get the two private keys, d_1 and d_2 . And once you have the private key you can decrypt the messages using $c^{d_1} \pmod{N_1}$ or $c^{d_2} \pmod{N_2}$.

Part B

Suppose there is a ballot initiative, a simple yes or no vote in your hometown. An RSA public, private key pair is generated of the form $(pk, sk) = ((N, e), d)$, and every vote is encrypted with the same key pair. These encrypted votes are of the form $[("yes")^e \pmod{N}, \text{NAME}]$ or $[("no")^e \pmod{N}, \text{NAME}]$. The vote itself is encrypted, but the name is not. They are sent over a public network to the vote counting server. Suppose you have access to every encrypted vote, including your own. Show this is insecure, and that you can tell exactly how everyone voted. Give a suggestion on how to fix it.

4.1 Part A 15 / 15

✓ - 0 pts (a) Correct (Stephen)

- 5 pts (a) Incorrect (Malav)

✓ - 0 pts (b) Correct answer (64) with complete justification.

- 2 pts (b) No simplification provided for mod

- 3 pts (b) Correct answer (64) with insufficient justification.

- 3 pts (b) Incorrect answer with significant progress

- 5 pts (b) Incorrect answer with little/no progress.

✓ - 0 pts (c) Fully Correct

- 1 pts (c) Minor Error

- 3 pts (c) Incomplete, insufficient, or partially correct answer

- 5 pts (c) Completely incorrect answer

Solution: Because you have access to all the votes including your own and since there were only two options, yes and no, and you know if you voted yes or no, you can therefore identify the cyphertext that corresponds to your own vote. If another person's vote has the same cyphertext as yours they voted for the same thing and if it doesn't match yours then they voted against you.

There are many ways that would add a layer of defense against this. The simplest would be to encrypt the names as well, so it would be much harder if not impossible to identify your own vote and identify your vote's cyphertext. Another solution would be instead of only encrypting the vote yes or no, add something else to the string before encryption like the person's name or their initials, anything that would make it so the cyphertexts would look totally different, but when decrypted it would still be clear what the vote would be.

4.2 Part B 5 / 5

✓ - 0 pts *Correct*

- 2 pts Incorrect justification for why the encryption is insecure.
- 2 pts Encryption suggestion too vague and can't be implemented
- 2.5 pts Missing suggestion
- 5 pts Completely incorrect / Missing

5.) Fast Fourier Transform (FFT) (10 points)

In this question, we'll see a basic example of using FFT to multiply two polynomials. Our polynomials here will be $f(x) = 10 + 3x + x^2$ and $g(x) = 1 - 3x$. Recall that $i = \sqrt{-1}$, which means that the 4th roots of unity are 1, i , -1 , and $-i$ (each of these complex numbers raised to the 4th power is 1). **Show your work for all parts.**

- (a.) Compute the FFT (i.e. the “value representation”) of $f(x)$ and $g(x)$ by evaluating them at the 4th roots of unity. You can just leave your answer as a tuple of complex numbers, for example, the FFT of $2 + x$ would be $(3, 2 + i, 1, 2 - i)$. (No need to do the actual recursive method, simply evaluating by hand is fine).
- (b.) Now, take the element-wise product of the two FFTs you computed above (that is, multiply element 1 of $\text{FFT}(f(x))$ with element 1 of $\text{FFT}(g(x))$, element 2 with element 2, etc.). Then, separately, compute the product of the two polynomials, $h(x) = f(x)g(x)$ (just with normal polynomial multiplication). Compute the FFT of $h(x)$. Does it match the element-wise product from earlier?

Solution: a. $f(x) = 10 + 3x + x^2$

$$x = 1 : 10 + 3 + 1 = 14$$

$$x = i : 10 + 3i - 1 = 9 + 3i$$

$$x = -1 : 10 - 3 + 1 = 8$$

$$x = -i : 10 - 3i - 1 = 9 - 3i$$

$$\text{FFT of } f(x) = (14, 9 + 3i, 8, 9 - 3i)$$

$$g(x) = 1 - 3x$$

$$x = 1 : 1 - 3 = -2$$

$$x = i : 1 - 3i = 1 - 3i$$

$$x = -1 : 1 + 3 = 4$$

$$x = -i : 1 + 3i$$

$$\text{FFT of } g(x) = (-2, 1 - 3i, 4, 1 + 3i)$$

b.

$$14 * -2 = -28$$

$$9 + 3i * 1 - 3i = 9 - 27i + 3i + 9 = 18 - 24i$$

$$8 * 4 = 32$$

$$11 - 3i * 1 + 3i = 11 + 33i - 3 + 9i = 8 + 42i$$

$$\text{FFT of } f(x) \text{ times FFT of } g(x) = (-28, 18 - 24i, 32, 18 + 24i)$$

$$h(x) = 10 + 3x + x^2 * 1 - 3x = 10 - 30x + 3x - 9x^2 + x^2 - 3x^3 = 10 - 27x - 8x^2 - 3x^3$$

$$x = 1 : 10 - 27 - 8 - 3 = -28$$

$$x = i : 10 - 27i + 8 + 3i = 18 - 24i$$

$$x = -1 : 10 + 27 - 8 + 3 = 32$$

$$x = -i : 10 + 27i + 8 - 3i = 18 + 24i$$

$$\text{FFT of } h(x) = (-28, 18 - 24i, 32, 18 + 24i) \text{ which is the same.}$$

5 Fast Fourier Transform (FFT) 10 / 10

✓ - 0 pts (a) Correct

$f: (14, 9 + 3i, 8, 9 - 3i)$

$g: (-2, 1 - 3i, 4, 1 + 3i)$

$h: (-28, 18 - 24i, 32, 18 + 24i)$

- 2 pts (a) FFTs correct; No work shown

- 2 pts (a) Work shown, but no final answer

- 2 pts (a) FFT wrong for $f(x)$

- 2 pts (a) FFT wrong for $g(x)$

✓ - 0 pts (b) Correct

$h(x) = 10 - 27x - 8x^2 - 3x^3$

$h: (-28, 18 - 24i, 32, 18 + 24i)$

- 3 pts (b) Answer is correct, but no work shown

- 2 pts (b) Answer is wrong, but consistent with Part A

- 2 pts (b) Work is correct, but final answer missing

- 7 pts Both (a) and (b) are wrong , but good attempt

- 10 pts Incorrect/Missing Answer

Problem Set 2: Divide & Conquer, RSA and FFT

Abraham Ladha

Due: 01/26/2023 11:59pm

- Please **TYPE** your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.
- Your solutions must be in plain English and mathematical expressions. Show the running time using the Master Theorem (wherever applicable).
- Unless otherwise stated, use $\log$ to the base 2.
- Unless otherwise stated, the fastest (and correct) algorithms will receive more credit. If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.
- Do not use pseudocode. Your answer will receive zero credit even if the pseudocode is correct.